The Networking Essentials Playbook

From Single Server to Kubernetes

by TechWorld with Nana

TECHWORLD WITH NANA

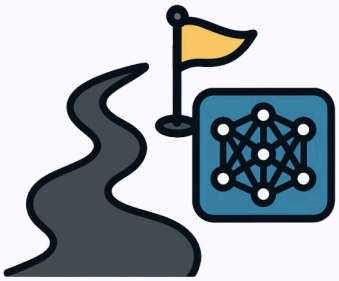
Welcome to Your Networking Journey!

I'm Nana, Co-Founder of TechWorld with Nana.

As a Cloud and DevOps engineer, I'm dedicated to helping engineers build the most valuable and highly-demanded DevOps and Cloud skills.

Through my [YouTube channel] and my comprehensive [DevOps bootcamps], I've helped 1,000,000s of engineers master the tools and concepts that drive modern software development.

About This Playbook



Follow one application's journey

- from a single server
- to a cloud-based Kubernetes cluster.

Learn exactly **why** each networking concept exists and how to use it in the real world.

This playbook gives you the knowledge to understand any networked system, at any scale.

Happy learning!

> **Nana Janashia**
> Co-Founder TechWorld with Nana

Core Networking Concepts

IP Addresses - Your Server's Home Address

Every device needs a unique identifier to receive data. Think of it like a house address for mail delivery.

Example:

- Public IP: 203.0.113.10 (visible on the internet)
- Private IP: 10.0.2.5 (only works inside your network)

Quick Tip: Private IPs save money and add security. They can't be reached directly from the internet.

DNS - The Internet's Contact List

Translates easy names into IP addresses so you don't memorize numbers.

You type: travelbuddy.com
DNS finds: 203.0.113.10
Browser connects automatically

Real-world use: Every website you visit uses DNS. Without it, you'd type 142.250.185.46 instead of google.com

Ports - Apartment Numbers for Applications

Multiple apps share one IP address. Ports (1-65,535) separate them.

Service	Port	
Website	80	Standard HTTP
Secure Website	443	HTTPS (encrypted)
Database	3306	MySQL default
Your Custom API	9090	You choose

Think of it as: One building (IP address), many apartments (ports)

© nnSoftwareGmbH - For more resources and tutorials, visit techworld-with-nana.com and our YouTube channel

Security & Network Organization

Subnets - Dividing Your Network into Zones

Frontend

10.0.1.x

→ Frontend (public-facing)

Application

10.0.2.x

→ Application servers (internal)

Database

10.0.3.x

→ Databases (most protected)

Why this matters: If hackers break into your website, they still can't reach your database.

Routing - GPS for Network Traffic

Routes determine the path data takes between network segments.

Simple rule: If two devices are in different subnets, they need a router to talk to each other.

Firewalls - Your Security Checkpoints

Control what traffic is allowed where. Two types you'll use:

Host Firewalls (on individual servers)

- Rule example: "Only accept database connections from frontend subnet"

Network Firewalls (between network segments)

- Rule example: "Allow incoming traffic on ports 80 and 443 only"

Pro tip: Use layers. Don't rely on one firewall - put them at multiple levels.

NAT - Share One Public IP with Many Servers

Lets private servers access the internet through one public IP address.

The problem: 50 backend servers need internet access, but public IPs cost money.

The solution: All 50 servers share 1 public IP through NAT.

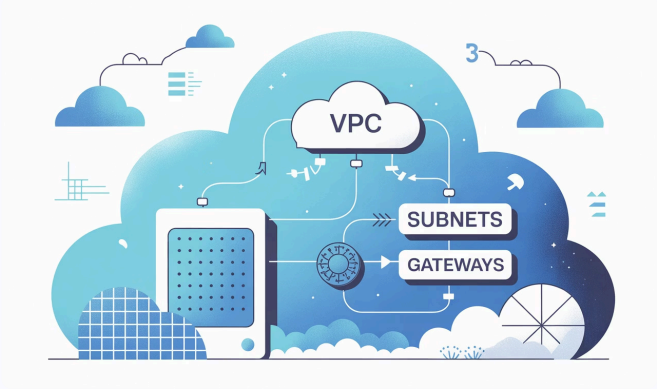
Like a receptionist: Employees make calls through the company's main number. The receptionist routes responses back to the right person.

© nnSoftwareGmbH - For more resources and tutorials, visit techworld-with-nana.com and our YouTube channel

Cloud Networking Essentials - 1

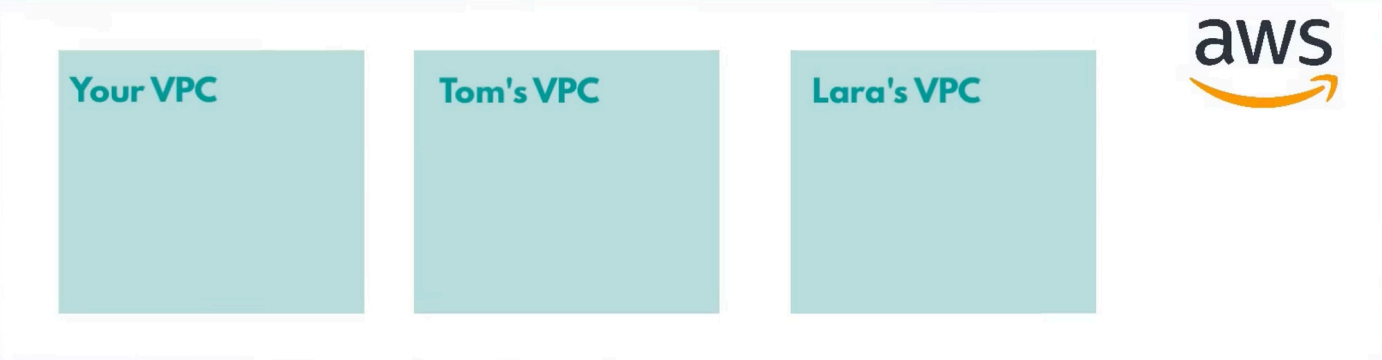
The networking concepts don't change in the cloud.

You still use IPs, ports, subnets, firewalls. They're just managed differently.



VPC - Your Private Space in the Cloud

Virtual Private Cloud = your isolated section of the cloud provider's network. For example in AWS cloud:



What you get:

- Your own IP address range
- Your own subnets
- Complete control over routing

© nnSoftwareGmbH - For more resources and tutorials, visit techworld-with-nana.com and our YouTube channel

Cloud Networking Essentials - 2

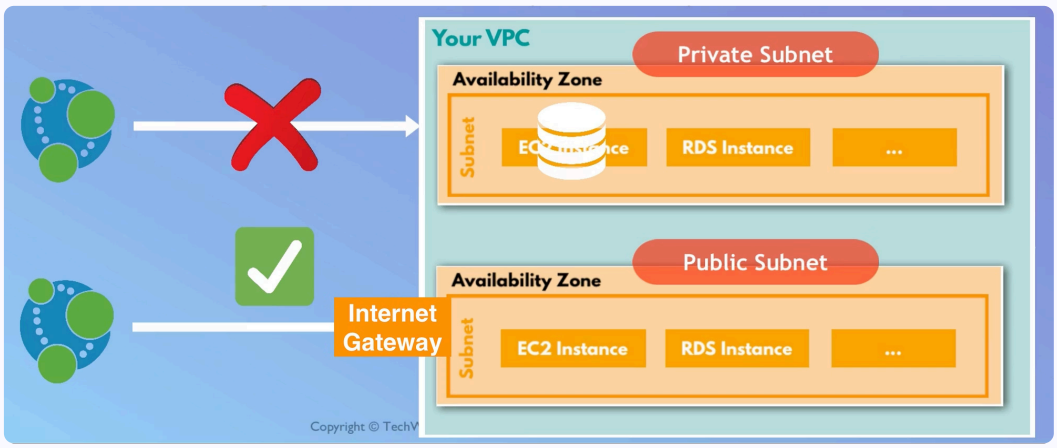
Internet Gateway

The main entrance connecting your VPC to the internet.

→ Internet

→ Internet Gateway

→ **Public** Subnet



NAT Gateway

Lets private subnet servers access the internet without being exposed.

Private Server → NAT Gateway (in public subnet) → Internet

Setup checklist:

1. Create NAT Gateway in public subnet
2. Point private subnet's route table to NAT Gateway
3. Private servers can now reach internet safely

Route Tables - Traffic Signposts

Tell traffic where to go. Each subnet has a route table.

Example Route Table:

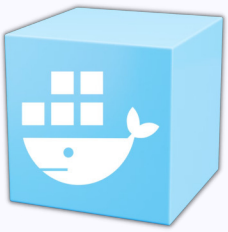
Destination: 0.0.0.0/0 → Target: Internet Gateway
(This means: send all internet traffic to the gateway)

© nnSoftwareGmbH - For more resources and tutorials, visit techworld-with-nana.com and our YouTube channel

Container Networking (Docker)

Container = Everything Your App Needs in One Package

Package your code, libraries, and settings together.



Runs the same everywhere:

- On laptop
- On development server
- On testing or production server

The power: "Works on my laptop" = "Works in production"

Bridge Network

Private network that exists on one server. Containers on the same bridge can talk to each other.

Container A → Bridge Network → Container B (They can use container names instead of IPs)

Port Mapping

Containers have internal ports. Map them to the host server.

```
docker run -p 9090:9090 myapp
```

Translation:

- Host port 9090 → Container port 9090
- Traffic to server's 9090 goes into container's 9090

Overlay Networks

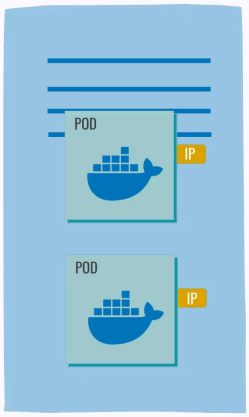
Connect containers across multiple servers. They act like they're on the same network even when on different hosts.

Use case: Your database container is on Server 1, your app container is on Server 2. Overlay network lets them talk seamlessly.

© nnSoftwareGmbH - For more resources and tutorials, visit techworld-with-nana.com and our YouTube channel

Kubernetes Networking - 1

Pods - The Basic Unit

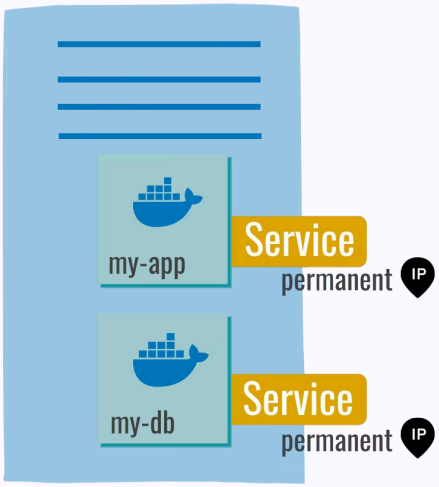


A pod wraps one or more containers. Usually just one container per pod.

Key fact: Each pod gets its own IP address.

The problem: Pods are temporary. They die and get recreated with new IPs constantly.

Services - Stable Endpoints for Unstable Pods



Provides a permanent IP and DNS name that never changes, even as pods come and go.

Why you need this:

Without Service: Website → Database Pod (IP changes = broken)

With Service: Website → Database Service → Database Pod (always works)

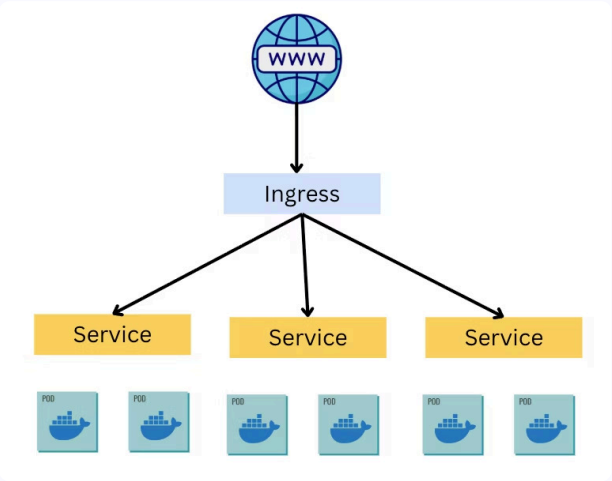
Service Types:

- **ClusterIP:** Only accessible inside the cluster (default)
- **NodePort:** Accessible from outside on a specific port
- **LoadBalancer:** Gets a real external IP (cloud providers)

© nnSoftwareGmbH - For more resources and tutorials, visit techworld-with-nana.com and our YouTube channel

Kubernetes Networking - 2

Ingress - The Smart Traffic Router



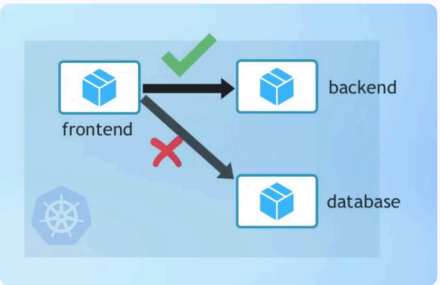
Routes external internet traffic to the right service inside your cluster.

One entry point, many services behind it.

travelbuddy.com/ → Website Service
travelbuddy.com/api/booking → Booking Service
travelbuddy.com/api/payment → Payment Service

Network Policies - Firewalls for Pods

Control which pods can talk to which other pods.



Example Policies:

- "Only pods labeled 'backend' can connect to pods labeled 'database'"
- "Pods labeled 'frontend' can only connect to pods labeled 'backend'"

ⓘ **Security win:** Even if one pod is compromised, the attacker can't reach other pods.

© nnSoftwareGmbH - For more resources and tutorials, visit techworld-with-nana.com and our YouTube channel

Troubleshooting & The Big Picture

📋 Troubleshooting Checklist

When something doesn't work, check these in order:

Can you find it?

Check DNS and IP addresses

Right port?

Verify the application is listening on the expected port

Can traffic reach it?

Check routing and route tables

Is it blocked?

Review firewalls, security groups, Network Policies

NAT issues?

Check if private servers can reach external services

> ⓘ **Pro debugging tip:** Use curl, ping, telnet, or kubectl exec to test each layer.

🧠 The Big Picture

These 5 concepts are everywhere:

Addressing

(IP, DNS) - How devices find each other

NAT

How private addresses access the internet

Ports

How multiple apps share one IP

Segmentation

(Subnets, Routing) - How we organize networks

Firewalls

How we control traffic

The tools change:

- Physical routers → VPCs
- Physical firewalls → Security Groups
- Bare metal → Containers → Kubernetes

But, the concepts stay the same.

Master these fundamentals and you'll understand networking at any scale - from a single server to a global Kubernetes cluster.

© nnSoftwareGmbH - For more resources and tutorials, visit techworld-with-nana.com and our YouTube channel

Ready to Future-Proof Your Career?

The truth is, your role today won't look the same in 3 years. Sys admins manage Kubernetes. Developers deploy their own code. Everyone needs to understand cloud infrastructure.

DevOps skills aren't just for "DevOps engineers"—they're for anyone building or running modern systems.

5,000+ students chose the structured path instead of wasting years piecing together random tutorials.

You can watch scattered YouTube videos for the next 3 years and still feel lost when it's time to actually deploy something.

Or you can follow a proven path and have real, job-ready skills in a few months.

> ⓘ **This is your last stop.** After this bootcamp, you won't need to hunt for more tutorials. You'll have the complete foundation to confidently work with modern infrastructure.
>
> The structured way saves you years of confusion. Learn more HERE

All the best on your journey! :)

Join 1.3 M+ engineers

Free DevOps and Cloud tutorials

Join 200k+ engineers

Behind-the-scenes and career content